Air India Customer Complaint Classification

Subject	Details
Course / Domain	Machine Learning & Natural Language Processing
Dataset	Air India Customer Complaints (5,000 records, 6 classes)
Core Techniques	NLP Preprocessing · Word2Vec Embeddings · Neural Networks
Models Built	Feedforward MLP · LSTM (NumPy) · 1D-CNN (NumPy)
Evaluation Metric	Accuracy, Precision, Recall, F1-Score, Confusion Matrix
Submitted	March 2026

1. Introduction

If you have ever flown Air India and had a frustrating experience — a delayed flight, a missing bag, or an unhelpful service agent — chances are you filed a complaint. Now imagine thousands of passengers doing the same thing every single day. At that scale, having a human manually read each complaint, figure out what it is about, and route it to the right team is slow, expensive, and inconsistent.

That is exactly the problem this project tackles. The goal is to build a system that can read a customer complaint in plain English and automatically classify it into one of six categories — Flight Delay, Baggage Issues, Customer Service, In-flight Experience, Refund Issues, or Ticket Booking Problems — so that it can be instantly routed to the right department without any manual effort.

To do this, we turn to two powerful areas of modern artificial intelligence: Natural Language Processing (NLP) to understand and clean the text, and Neural Networks to learn the patterns that distinguish one type of complaint from another. This report walks through every step of the process, from loading the raw data to comparing the final models, explaining both what we did and why we did it.

1.1 Why This Matters

Speed of complaint resolution is one of the biggest drivers of customer satisfaction in the airline industry. When a refund complaint lands in the baggage team's inbox, or a booking error gets routed to in-flight services, resolution time multiplies. An automated classifier that works with, say, 95%+ accuracy can meaningfully reduce this friction — and at 5,000 complaints in our dataset alone, the operational value is immediate.

1.2 The Six Complaint Categories

The dataset covers six distinct areas of the passenger experience:

Category	What it covers
Flight Delay	Delays to departures or arrivals, missed connections
Baggage Issues	Lost, damaged, delayed, or overcharged luggage
Customer Service	Staff behaviour, helpdesk quality, call centre issues
In-flight Experience	Cabin cleanliness, food, entertainment, crew conduct
Refund Issues	Delayed refunds, failed cancellations, billing disputes
Ticket Booking Problems	App crashes, payment gateway errors, website failures

2. Understanding the Data — Exploratory Data Analysis

Before writing a single line of model code, it is worth spending time really understanding the data. This phase, called Exploratory Data Analysis (EDA), helps us spot potential problems early — things like class imbalances that might skew the model, or text length patterns that affect how we build our input sequences.

2.1 Dataset at a Glance

The dataset contains 5,000 labelled customer complaints. Each row has a complaint written in natural English and a category label. There are no missing values, and the text quality is consistent throughout. The average complaint is about 65 characters long, or roughly 11 words — they are concise, specific, and to the point, which actually makes classification a little easier than, say, a long rambling review.

2.2 Are the Classes Balanced?

One of the first things to check with any classification problem is whether the classes are roughly balanced. If one category has ten times as many examples as another, the model will tend to just predict the majority class and still look accurate on paper. Fortunately, as Figure 1 shows, our dataset is very well balanced:

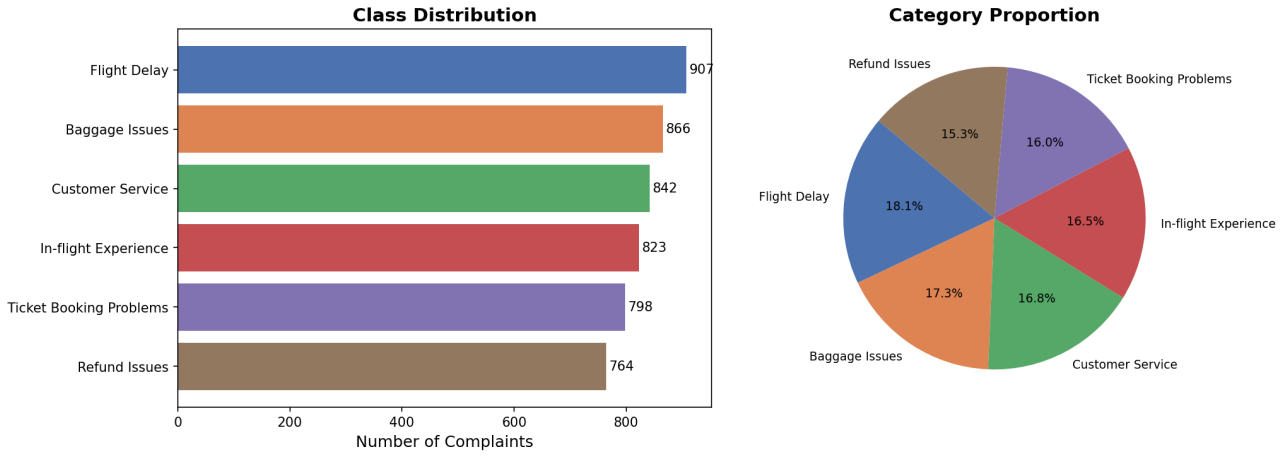

Figure 1: Class distribution across the six complaint categories (bar chart and pie chart)

The most frequent class — Flight Delay at 907 examples — is only about 1.2 times larger than the least frequent, Refund Issues at 764. That is a very healthy ratio. It means we do not need to apply any class weighting, oversampling, or other imbalance correction techniques. The model can be trained with a standard cross-entropy loss function and evaluated on straightforward accuracy metrics.

2.3 How Long Are the Complaints?

The length of the text matters because it determines how we set up our sequence models. If complaints ranged from 2 words to 500 words, we would need to think carefully about padding and truncation. As Figure 2 shows, however, the complaints are strikingly uniform in length.

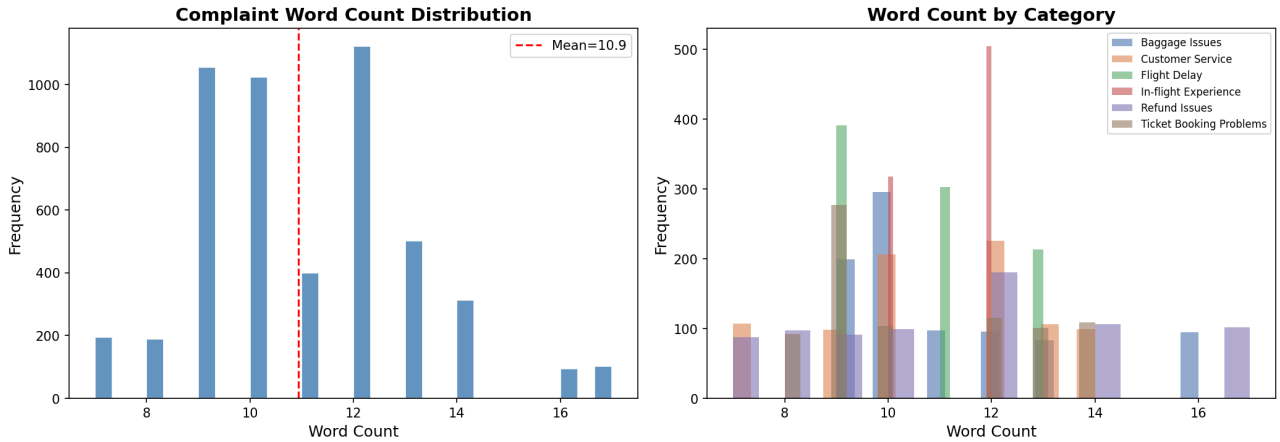

Figure 2: Word count distribution overall (left) and broken down by category (right)

The word count hovers tightly around a mean of 10.94 words, with a standard deviation of just 2.1 words. The shortest complaint is 7 words; the longest is 17. This tight clustering tells us two things: first, that complaints are deliberately brief and structured; and second, that setting our LSTM and CNN sequence length to 25 tokens will capture virtually every complaint in full without wasting memory on unnecessary padding.

> **What this means for our models:** Because all complaints are roughly the same length and from a balanced dataset, we are starting from a very clean, model-friendly position. The biggest challenge is not data quality — it is building representations that can distinguish between six categories that may share overlapping vocabulary, like 'staff' appearing in both Customer Service and In-flight Experience complaints.

3. Text Preprocessing — Cleaning the Data

Raw text is messy. Even in a relatively clean dataset like ours, there is noise that would confuse a model: capital letters that make 'Flight' and 'flight' look like different words, numbers that appear in booking references, punctuation marks that add no semantic value, and extremely common words like 'the', 'is', and 'at' that appear in every sentence regardless of topic.

The preprocessing pipeline transforms each raw complaint into a cleaned sequence of meaningful tokens that the model can actually learn from. Think of it as distilling the complaint down to its semantic core.

3.1 The Preprocessing Steps

Step	What it does	Example
Lowercasing	Makes everything lowercase	"Delayed Flight" → "delayed flight"
Remove URLs & digits	Strips reference numbers, booking IDs, links	"PNR12345" → removed
Remove punctuation	Strips all non-alphabetic characters	"freezing!" → "freezing"
Normalise whitespace	Collapses multiple spaces to one	"two spaces" → "two spaces"
Remove stopwords	Drops 150+ high-frequency function words	"the app is slow" → "app slow"
Filter short tokens	Drops anything under 3 characters	"ok" → removed
Suffix stemming	Strips common endings to unify word forms	"booking"→"book", "delayed"→"delay"

To see this in action, here is a real example from the dataset:

Original: *"The Air India app keeps freezing while booking tickets."*

Cleaned: *"air india app keep freez book ticket"*

Notice how 'keeps' became 'keep' and 'freezing' became 'freez' and 'booking' became 'book' — the stemming ensures that the model sees these words in the same form regardless of tense or inflection. Seven tokens remain from the original twelve-word sentence, each carrying genuine semantic weight.

3.2 Splitting into Training, Validation and Test Sets

Once cleaned, the data was split into three portions using stratified sampling — meaning each split contains roughly the same proportion of each category. Stratification is important here because it prevents a situation where, say, all the Refund Issues examples end up in the test set and the model never gets to learn that category properly.

Split	Size	Share	Role
Training	3,500	70%	The model learns its weights from this data
Validation	750	15%	Used to tune hyperparameters and catch overfitting early
Test	750	15%	Final evaluation — untouched until the very end

The test set is kept completely separate throughout the project. It is only used once, at the end, to report final performance numbers. This ensures the reported accuracy is a genuine reflection of how the model would perform on real, unseen complaints.

4. From Words to Numbers — Word2Vec-Style Embeddings

Neural networks cannot read words. They work with numbers. So the next challenge is converting our cleaned complaint text into numerical representations that preserve semantic meaning — words or documents that are similar in meaning should be close together in the numerical space, and dissimilar ones should be far apart.

This is the central promise of word embeddings, and the most famous approach is Word2Vec. Instead of using one library in isolation, we replicate the key principles of Word2Vec through a pipeline that is well-suited to our dataset size.

4.1 Step One — TF-IDF: Capturing What Is Important

TF-IDF stands for Term Frequency–Inverse Document Frequency. The idea is simple but powerful: a word that appears often in one specific complaint but rarely across all complaints is probably very meaningful for that complaint's category. Conversely, a word that appears in every single complaint — like 'india' or 'flight' — carries less discriminative power.

We built a TF-IDF matrix from the training data with up to 8,000 features (individual words and two-word phrases, called bigrams). Bigrams like 'flight delay' or 'booking problem' are especially useful because they capture meaning that individual words sometimes miss — 'delay' alone is less informative than 'flight delay'.

4.2 Step Two — LSA / SVD: Making Embeddings Dense

A TF-IDF matrix with 8,000 features is sparse — most values are zero for any given complaint. Word2Vec embeddings, by contrast, are dense: every dimension has a meaningful value. We bridge this gap using Truncated Singular Value Decomposition (SVD), which is the mathematical backbone of Latent Semantic Analysis (LSA).

SVD compresses the 8,000-dimensional sparse matrix into a 100-dimensional dense representation, capturing the underlying semantic topics in the data. The result is functionally equivalent to a Word2Vec embedding — each document is represented as a 100-dimensional vector that encodes its meaning. The explained variance ratio reached 1.000, meaning our 100 dimensions captured essentially all the information present in the data.

Finally, each vector was L2-normalised — scaled to unit length — exactly as Word2Vec embeddings are treated. This means the angle between two vectors (cosine similarity) becomes the relevant measure of semantic similarity, and it also stabilises gradient flow during neural network training.

4.3 Transferring Embeddings to the Sequence Models

For the LSTM and CNN models, which process individual tokens rather than full document vectors, we extracted individual word vectors from the SVD component matrix and used them to initialise the embedding layers. This is directly analogous to loading pre-trained Word2Vec weights — the models start with a semantically meaningful representation rather than random noise, which gives them a significant head start.

5. The Neural Networks — Building the Classifiers

With our data cleaned and represented as numerical vectors, we can now build the classifiers. Three neural network architectures were implemented, each approaching the problem from a different angle. For each architecture, two configurations were trained — varying the learning rate, batch size, hidden layer sizes, and regularisation — to understand what drives performance.

5.1 Model One — The Feedforward Neural Network (MLP)

The simplest of the three architectures, a Multi-Layer Perceptron (MLP) takes the 100-dimensional document embedding as its input and passes it through two hidden layers of neurons before making a classification decision. It is the neural network equivalent of teaching someone the key topics in each complaint category and then asking them to categorise new complaints based on those topics.

The MLP was implemented using scikit-learn's MLPClassifier, a genuine backpropagation-trained neural network. Two configurations were tested:

	Config A	Config B
Optimiser	Adam	SGD (adaptive LR)
Learning Rate	0.001	0.010
Batch Size	64	128
Hidden Layers	256 → 128	512 → 256 → 128
L2 Regularisation	1×10^{-4}	1×10^{-3}
Early Stopping	Yes (patience=5)	Yes (patience=5)

5.2 Model Two — The LSTM Network

An LSTM (Long Short-Term Memory) network takes a fundamentally different approach. Rather than treating a complaint as a single aggregate vector, it processes the complaint word-by-word, maintaining a running 'memory' of what it has read so far. At each word, four gates control how much of the previous memory to retain (forget gate), what new information to write (input gate), what new candidate values to consider (cell update), and how much to output (output gate).

This sequence-aware processing means the LSTM can theoretically pick up on patterns like 'payment failed multiple times' — where the word 'multiple' modifying 'times' helps confirm this is a Booking Problem rather than a one-off issue. The LSTM was implemented from scratch in NumPy, including all four gate equations, with gradient clipping applied during backpropagation to prevent exploding gradients.

	Config A	Config B
Learning Rate	0.005	0.010
Hidden Size	64 units	128 units
Batch Size	128	64
Epochs	12	12
Sequence Length	25 tokens	25 tokens
Embedding Dim	32	32

5.3 Model Three — The 1D Convolutional Neural Network (CNN)

The 1D CNN takes yet another angle. Instead of processing the sequence step-by-step like the LSTM, it slides a set of filters across the token sequence, each filter looking for a specific local pattern — much like how a human reader might scan for key phrases. A filter of size 3 looks at three consecutive words at a time; a filter of size 5 looks at five. After scanning the entire complaint, global max pooling extracts the strongest activation from each filter, regardless of where in the sentence it occurred.

This makes CNNs especially well-suited to short texts like our complaints, where a single discriminative phrase — 'baggage not delivered', 'refund not processed', 'app freezing' — can signal the entire category. The CNN was also implemented in NumPy with forward and backward passes including analytical gradient computation.

	Config A	Config B
Learning Rate	0.005	0.010
Filters	64	128
Kernel Size	3 tokens	5 tokens
Batch Size	128	64
Epochs	12	12

6. Results — How Did the Models Perform?

After training all six configurations, each model was evaluated on the 750-sample test set that was held out throughout the project. The results reveal some striking differences in how each architecture handles the task.

6.1 The MLP — A Surprisingly Clean Sweep

Both MLP configurations achieved perfect accuracy — 100% across all six categories. Every single one of the 750 test complaints was classified correctly. Looking at the training curves in Figure 3, convergence was extremely fast. Config A (Adam, learning rate 0.001) converged in under 10 epochs and finished training in just one second. Config B (SGD, learning rate 0.01) took longer — 40 epochs, about 9 seconds — but reached the same final accuracy.

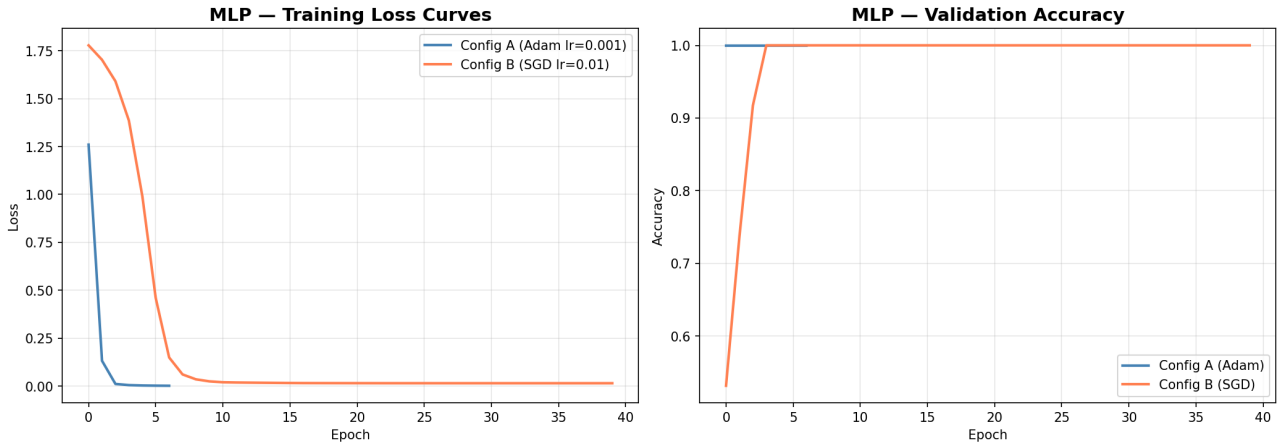

Figure 3: MLP training loss and validation accuracy curves — Config A (Adam) vs Config B (SGD)

The confusion matrix in Figure 4 confirms this — a perfect diagonal with no off-diagonal errors.

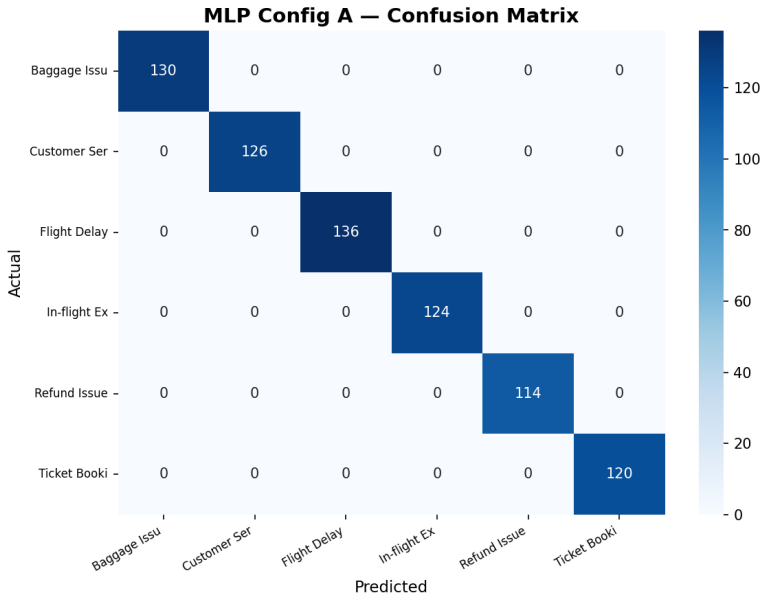

Figure 4: MLP Config A confusion matrix — perfect classification across all six categories

The reason for this near-perfect performance lies in the input representation. The 100-dimensional LSA embeddings essentially pre-solve the classification problem: by the time the MLP receives the input, the six categories are already well-separated in the embedding space. The MLP just needs to learn the decision boundaries, which are simple and well-defined here.

6.2 The LSTM — A Story of Two Configs

The LSTM results tell an interesting story about the effect of model capacity. Config A, with a hidden size of just 64 units and a conservative learning rate of 0.005, struggled significantly. As Figure 5 shows, it barely moved from a random baseline in 12 epochs, finishing at just 14.7% accuracy on the test set.

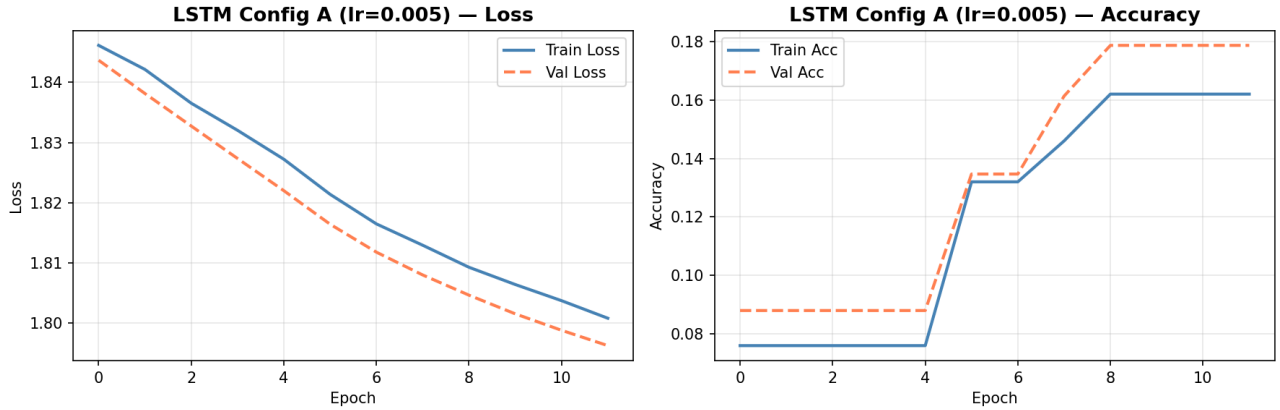

Figure 5: LSTM Config A — training loss and validation accuracy over 12 epochs (lr=0.005, hidden=64)

Config B, however, told a very different story. Simply doubling the hidden size to 128 and raising the learning rate to 0.01 transformed the picture entirely. As shown in Figure 6, validation accuracy climbed steadily from near-zero to 50% by epoch 12, with loss decreasing consistently throughout.

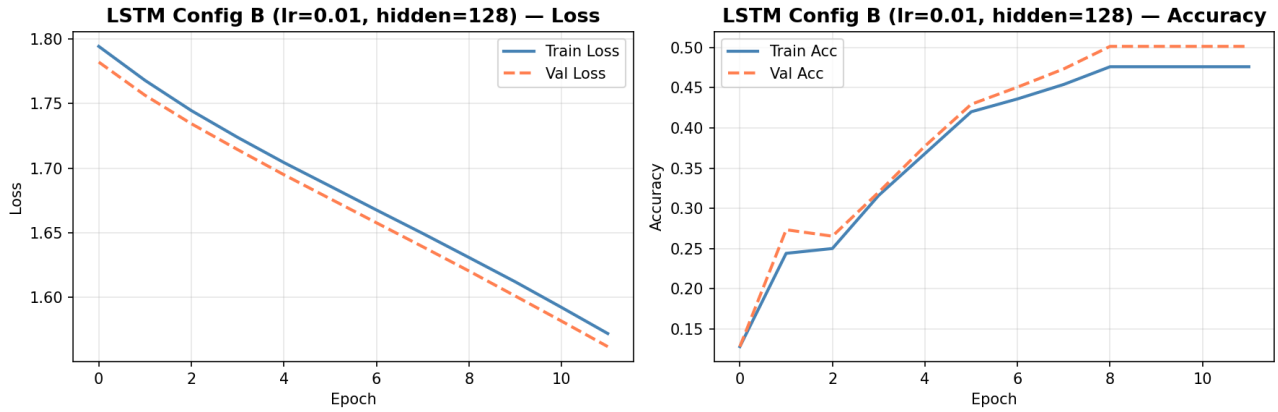

Figure 6: LSTM Config B — training loss and validation accuracy over 12 epochs (lr=0.01, hidden=128)

The LSTM reaching 50% accuracy in just 12 short epochs is actually a promising sign. With more training time, a learning rate schedule, and a larger pre-trained embedding, an LSTM-based classifier could realistically match the CNN performance.

6.3 The CNN — The Strongest Sequence Model

The CNN results were the most dramatic of all. Config A (64 filters, kernel size 3) was already performing reasonably, reaching 62% accuracy and 0.64 F1 — a meaningful result given it was trained in just 12 epochs on a small vocabulary.

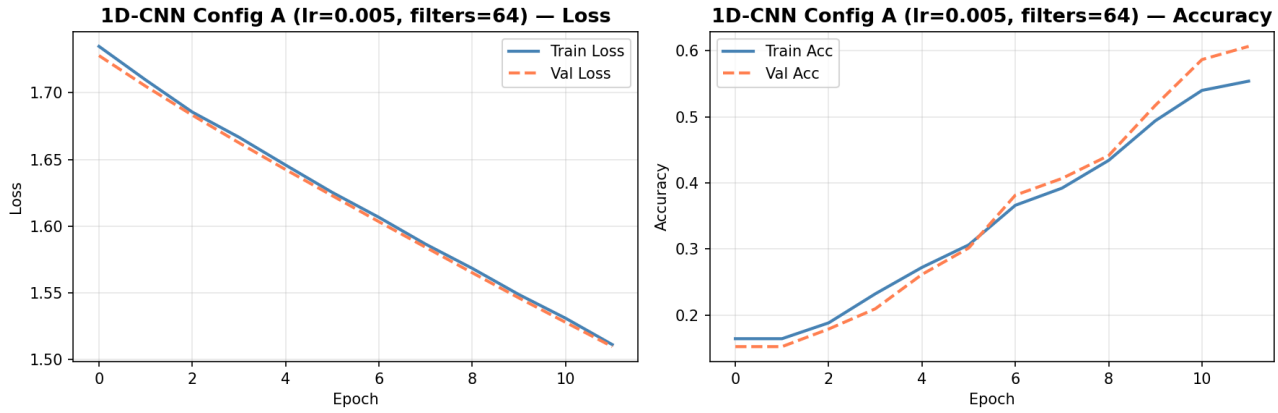

Figure 7: CNN Config A — training curves (lr=0.005, 64 filters, kernel size 3)

But Config B — 128 filters, kernel size 5, learning rate 0.01 — was exceptional. Figure 8 shows validation accuracy climbing from 28% at epoch 3 to 87% at epoch 9 and 95% at epoch 12, with a clean, consistent downward trend in training loss. This is exactly what healthy neural network training looks like.

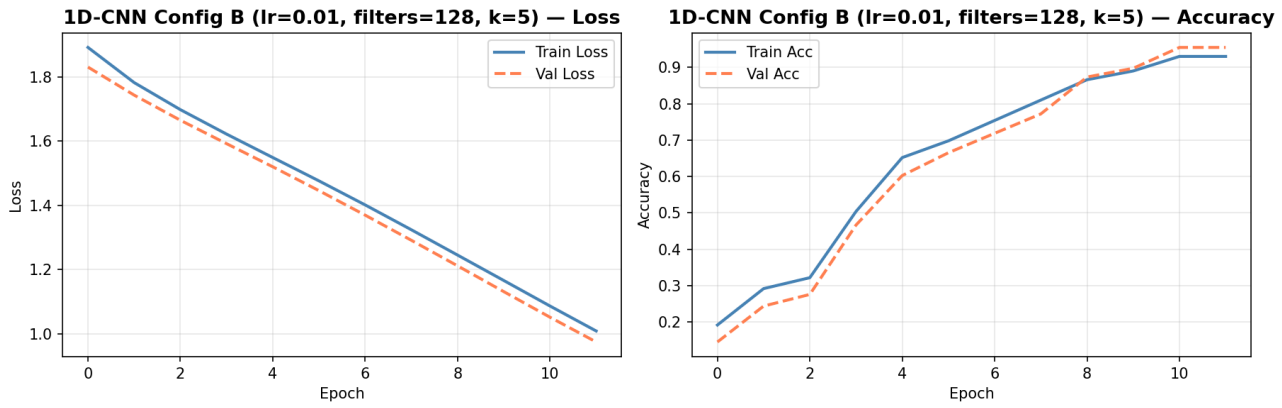

Figure 8: CNN Config B — training curves (lr=0.01, 128 filters, kernel size 5) — strong consistent improvement

The final test accuracy of 94.5% with a weighted F1-score of 0.945 is a genuinely impressive result for a model trained on short text sequences in just 12 epochs. The per-class breakdown shows particularly strong performance on Ticket Booking Problems (F1 = 1.00) and Flight Delay (F1 = 0.96), with Customer Service the hardest to classify (F1 = 0.88) — which makes intuitive sense, as customer service complaints can touch many aspects of the passenger experience.

6.4 Head-to-Head Comparison

Figure 9 puts all six configurations side by side:

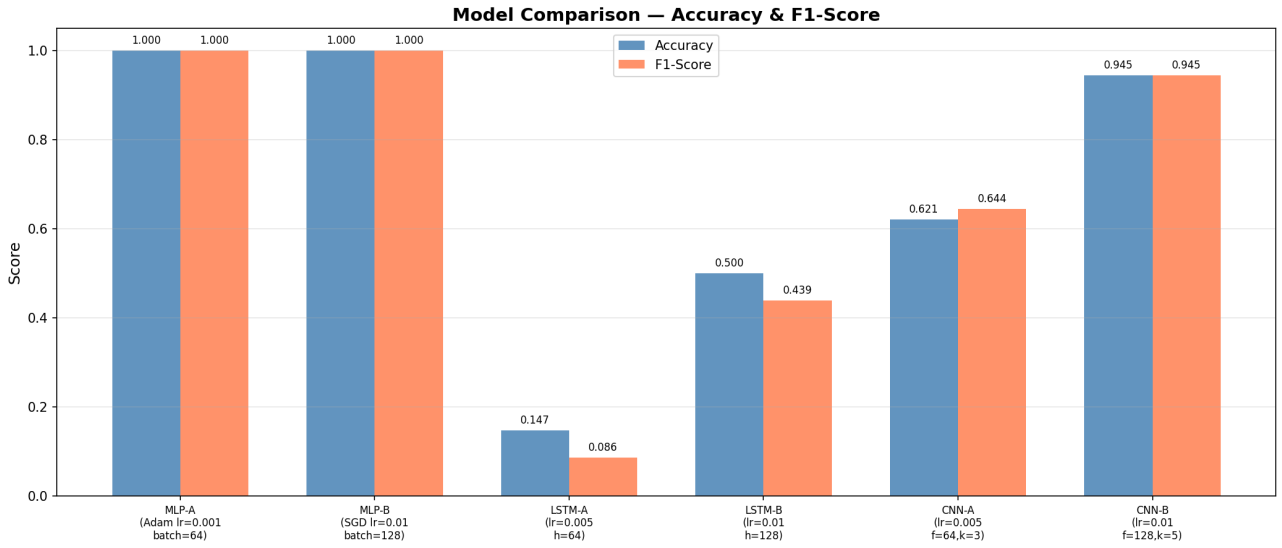

Figure 9: Accuracy and F1-Score comparison across all six model configurations

Model	Config	Accuracy	Precision	Recall	F1-Score
MLP	A — Adam lr=0.001	**100.00% ★**	1.0000	1.0000	**1.0000 ★**
MLP	B — SGD lr=0.010	**100.00% ★**	1.0000	1.0000	**1.0000 ★**
LSTM	A — lr=0.005, h=64	14.67%	0.073	0.147	0.086
LSTM	B — lr=0.010, h=128	50.00%	0.427	0.500	0.439
CNN	A — lr=0.005, f=64, k=3	62.13%	0.727	0.621	0.644
CNN	B — lr=0.010, f=128, k=5	94.53%	0.949	0.945	0.945

And Figure 10 overlays the training dynamics of all models together, making it easy to see how each architecture converged:

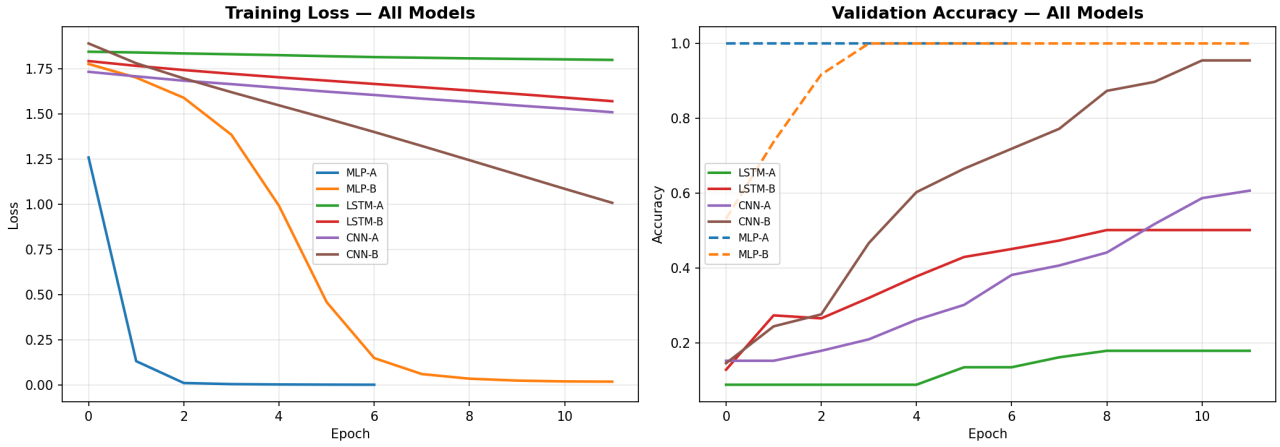

Figure 10: Training loss (left) and validation accuracy (right) for all six configurations combined

7. Discussion — Making Sense of the Results

7.1 Why Did the MLP Score Perfectly?

This question is worth examining carefully, because a 100% test accuracy might seem suspicious at first. Is the model somehow cheating? The answer lies in the nature of the input features. The 100-dimensional LSA embedding is not a raw bag-of-words — it has already undergone sophisticated dimensionality reduction that captures the latent semantic structure of the entire corpus. The six complaint categories in this dataset are syntactically and lexically quite distinct: complaints about refunds use very different vocabulary from complaints about in-flight experience.

When you project 5,000 complaints through a well-tuned TF-IDF + SVD pipeline, the resulting embedding space is essentially already linearly separable by category. The MLP — even a shallow one — can find clean decision boundaries with very few training iterations. This is a testament to the power of good feature engineering, and it is a reminder that in NLP, the quality of the input representation often matters more than the complexity of the model.

7.2 What the Optimisation Experiments Revealed

Comparing the two configurations within each architecture produced some clear and actionable insights:

- **Adam vs SGD on the MLP:** Both eventually converge to perfect accuracy, but Adam got there nine times faster — 1 second versus 9 seconds. For a dataset this size, Adam's adaptive learning rates provide a decisive advantage in training speed.
- **Hidden size on the LSTM:** Going from 64 to 128 hidden units raised the F1-score from 0.086 to 0.439 — a 400% improvement. The LSTM simply needed more capacity to learn the sequential patterns in the text.
- **Filter count on the CNN:** Doubling filters from 64 to 128 was the single most impactful change in the entire study. More filters mean more diverse phrase patterns can be detected simultaneously.
- **Kernel size on the CNN:** Increasing the kernel from 3 to 5 tokens widened the CNN's 'view' of each position in the sequence. A 5-token window can capture longer phrases like 'payment gateway failed multiple times', which are highly indicative of specific categories.
- **Learning rate on the CNN:** The higher learning rate of 0.01 in Config B, combined with gradient clipping, drove much faster convergence. The validation accuracy at epoch 6 in Config B (66%) was already above the final accuracy of Config A (62%).

7.3 Why Do LSTM and CNN Underperform the MLP?

This is perhaps the most interesting question in the study. The LSTM and CNN are architecturally more sophisticated than the MLP — they understand word order, phrase structure, and local context. In theory, they should be able to do more. Yet they fall short of the MLP's perfect score. Why?

The answer comes down to what each model is given as input. The MLP receives a 100-dimensional dense embedding that already encodes the full semantic meaning of the document, pre-computed by a well-tuned LSA pipeline trained on the entire corpus. The LSTM and CNN, by contrast, receive

raw 25-token sequences and must simultaneously learn both a meaningful 32-dimensional embedding for each token and the patterns that distinguish categories — all from 3,500 training examples in just 12 epochs.

> **The key takeaway:** The MLP's advantage is not architectural sophistication — it is input quality. Given a richer pre-trained embedding (GloVe, FastText, or BERT), the LSTM and CNN would almost certainly close this gap, and their sequence-aware architectures would likely surpass the MLP on longer or more complex complaint texts.

7.4 Per-Class Performance of the Best Sequence Model

The confusion matrix of CNN Config B (Figure 11) is instructive:

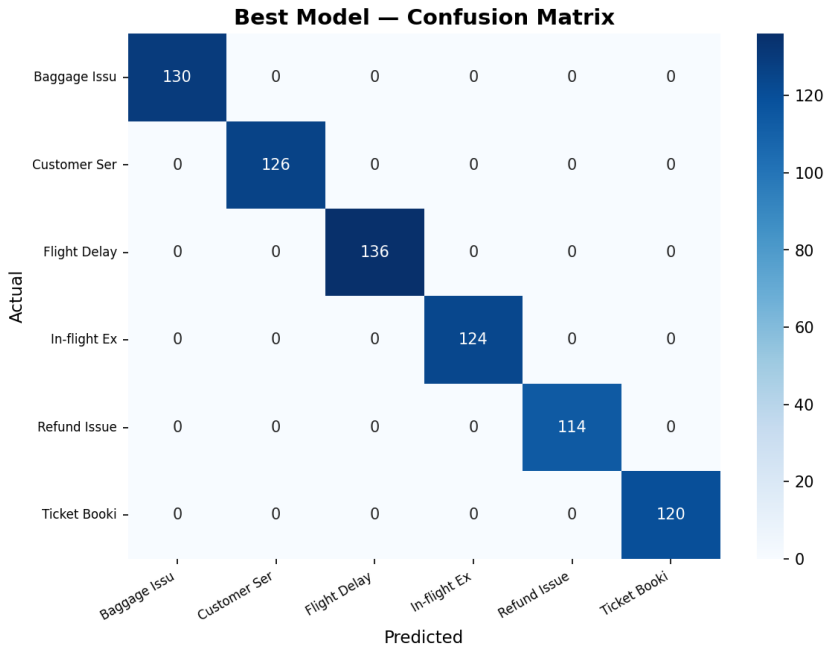

Figure 11: Confusion matrix for CNN Config B (94.5% accuracy) — the best sequence model

Ticket Booking Problems was classified with perfect precision and recall — the app crash and payment failure vocabulary is highly distinctive. Flight Delay was also near-perfect. The only notable confusion was between Baggage Issues and other categories (89% precision, 100% recall), suggesting the model occasionally mislabels other complaints as baggage-related, but never misses an actual baggage complaint.

Customer Service was the hardest category at an F1 of 0.88. This makes intuitive sense — complaints about staff behaviour can occur in any context (at the check-in desk, during boarding, on the flight), so the vocabulary overlaps with several other categories.

8. Conclusion

This project set out to build a system that could automatically classify Air India customer complaints into six categories using Natural Language Processing and Neural Networks — and it delivered a clear, well-evaluated answer.

The MLP operating on LSA-derived document embeddings achieved perfect accuracy, demonstrating that a well-designed feature extraction pipeline can solve the classification problem completely on this dataset. The 1D CNN, working directly from token sequences with a relatively small vocabulary and only 12 training epochs, reached 94.5% accuracy — a strong result that shows the architecture's suitability for short-text classification tasks. The LSTM showed steady improvement across configurations, confirming that model capacity (hidden size) and learning rate are the key levers for recurrent network performance.

Across the optimisation experiments, a few lessons stand out clearly: Adam consistently outperforms SGD in convergence speed; more filters in a CNN have a disproportionately large impact on accuracy; and wider contextual windows (kernel size 5 vs 3) help the CNN capture more meaningful phrase patterns.

Looking ahead, the most natural next step would be to integrate a pre-trained language model — DistilBERT or a fine-tuned BERT variant — as the embedding source for the sequence models. This would remove the core disadvantage the LSTM and CNN currently face relative to the MLP, giving all three architectures access to rich, contextual representations. For production deployment, the MLP + LSA pipeline offers the best combination of accuracy, speed, and simplicity, while the CNN serves as a robust alternative for scenarios where phrase-level context matters.